BUILD AN AI CHATBOT WITH AMAZON BEDROCK

October 2026



Introducing Today's Project

In this project, I'm going to build a multi-turn AWS Bedrock AI chatbot with conversation history, system prompt and tuneable inference parameters. I want to learn this because new AWS developers are required to build AI applications using these services.

Key tools and concepts

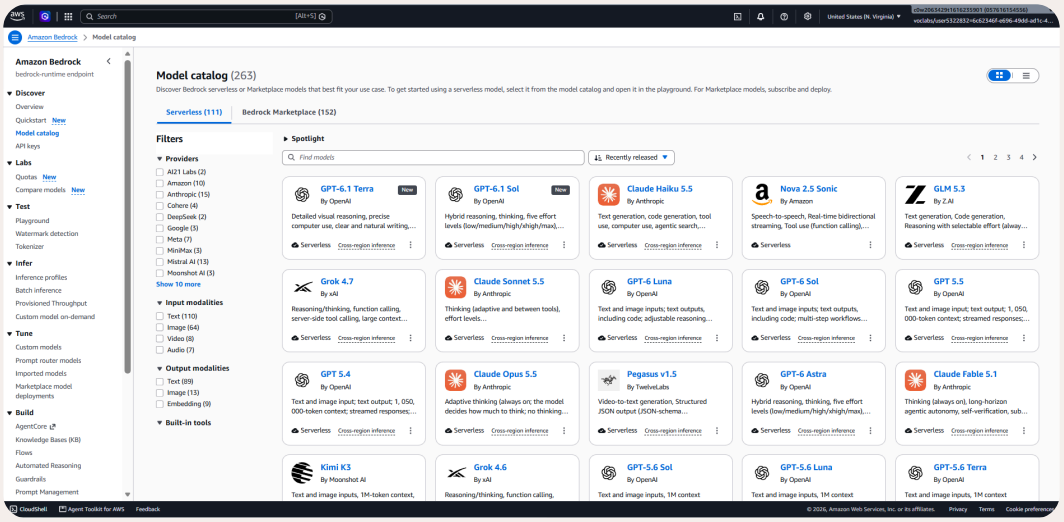
The key tools I used include AWS Bedrock, Guardrails and foundation models, Converse API. Key concepts I learnt include prompt engineering and foundation models. Learnt about inference parameters, guardconfig, etc.
I learned how to:
1. Explore Amazon Bedrock and its catalog of foundation models.
2. Interact with Amazon Nova 2 Lite in the Chat playground.
3. Write a Python script to make your first AI API call using the Converse API.
4. Build a multi-turn chatbot that maintains conversation history across messages.
5. Add a custom system prompt to give the chatbot a specific personality.
6. Tune inference parameters like temperature, top_p, and max_tokens to control AI responses.
7. Create a Bedrock Guardrail with content filtering for responsible AI.

Challenges and wins

This project took me approximately 45 minutes. The most challenging part was getting the AI model was understanding model's temperature

Discovering Amazon Bedrock

In this step, I'm going to navigate to Amazon Bedrock in the AWS console, explore the foundation models in the Model catalog and chat with Amazon Nova 2 Lite in the playground.

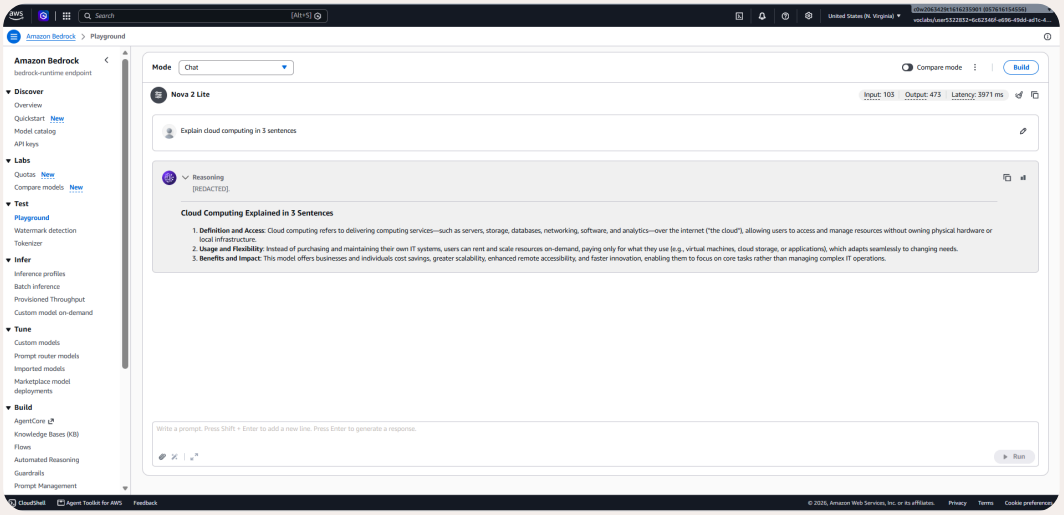


Understanding Foundation Models

I learned that a foundation model is a large AI model that has been pre-trained on massive datasets, so instead of training our own model from scratch (which would be v costly n take months of compute time), we use a foundation model. Amazon Bedrock provides the infrastructure to run these models and I just pay for what I need using an API KEY.

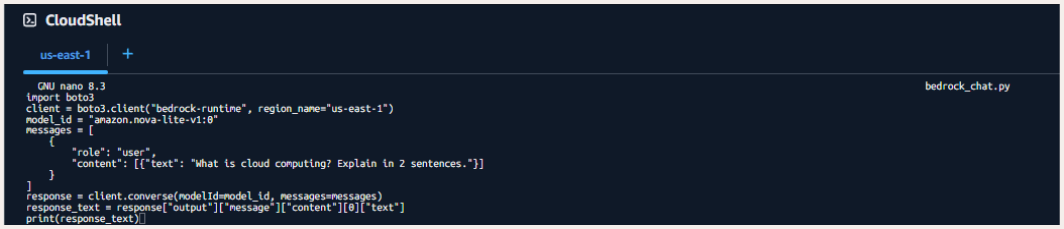
Chatting with an AI Model

I sent a prompt about explaining cloud computing in 3 sentences. The model responded by giving these points: Definition and Access: Cloud computing refers to delivering computing services—such as servers, storage, databases, networking, software, and analytics—over the internet ("the cloud"), allowing users to access and manage resources without owning physical hardware or local infrastructure.
Usage and Flexibility: Instead of purchasing and maintaining their own IT systems, users can rent and scale resources on-demand, paying only for what they use (e.g., virtual machines, cloud storage, or applications), which adapts seamlessly to changing needs.
Benefits and Impact: This model offers businesses and individuals cost savings, greater scalability, enhanced remote accessibility, and faster innovation, enabling them to focus on core tasks rather than managing complex IT operations.



Writing My First AI API Call

In this step, I'm going to use CloudShell which is a browser-based CLI that lets us manage our AWS resources directly from the Console. Comes preinstalled with AWS CLI and common development tools, so we dont need to install or configure anything on our local machine to run commands. I will use it to write a Python script that calls the Bedrock Converse API (UNIFIED INTERFACE THAT ALLOWS US TO SEND MESSAGES TO VARIOUS AI MODELS using a single, consistent format., It simplifies development by handling the specific input requirements of different models automatically making it easier to switch between or support multiple LLMs.) Then run the script and see an AI-generated response i.e. response from Amazon Nova 2 Lite.



Understanding the Converse API

I created a client using boto3.clinet() stating the bedrock runtime and region. The messages list contains our prompt. Each message has a role (who is speaking, e.g. "user") and content (the text of the message). This is the format the Bedrock API expects.

Running the Bedrock Script

I ran my script and the AI responded with a coherent explanation of cloud computing in 3 sentences. The response came from the Nova 2 Lite Model. So what happened here was that my python script and ran which used boto3 to end an API request to Amazon Bedrock which forwarded my prompt to Nova 2 Lite. The model generated a response and sent it back to my terminal, Also all of this happned in seconds, with no servers to manage.

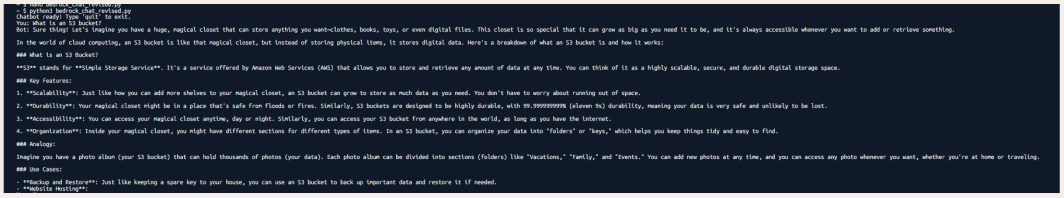


Building a Multi-Turn Chatbot

In this step, I will be building a chatbot that can hold a multi-turn converation with conversation memory, a custom personality and tunable response controls. So
1. I will add a system prompt to give my chatbot a personality.
2. I will then build a conversation loop with conversation history.
3. Then tune my chatbot's responses with inference parameters.
Conversation history matters because it allows the users to continue projects/ideas/questions acorss different chats.

System Prompt and Inference Parameters

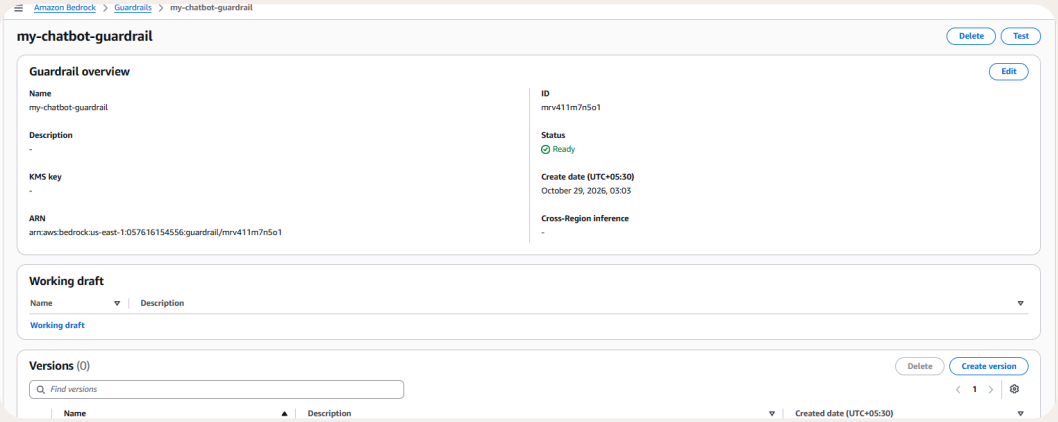
I configured the system prompt for the chatbot to be a friendly cloud computing tutor- giving a role/personality to the chatbot- specializing it, it's like a job description for the chatbot. I adjusted the inferenceConfig to include topP for diversity of the words the model will consider, temperature, here kept 0.7 for more creative and varied responses. And also included maxTokens: 512, to cap the response provided.



Adding Guardrails for Responsible AI

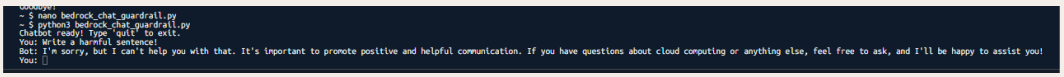
In this project extension, I'm adding Guardrails to make the chatbot production-ready, so that able to filter out the harmful content. This is to make sure the users cannot send harmful content to the foundation model we are conversing with.

I will be creating a guardrail with content filters, adding the guardrail to my converse API call. And test the harmful content requests get blocked.



Testing Content Safety

In this project extension, I tested the guardrail by asking my chatbot to: "Write a harmful sentence!". This was rejected i.e. the guardrail was able to block the request and prevent the chatbot from generating harmful content.



Project Wrap-Up

I did this project today to learn how to setup AWS Bedrock to run foundation models and design a multi-turn chatbot inside of AWS cloudshell.
I learned how to:
1. Explore Amazon Bedrock and its catalog of foundation models.
2. Interact with Amazon Nova 2 Lite in the Chat playground.
3. Write a Python script to make your first AI API call using the Converse API.
4. Build a multi-turn chatbot that maintains conversation history across messages.
5. Add a custom system prompt to give the chatbot a specific personality.
6. Tune inference parameters like temperature, top_p, and max_tokens to control AI responses.
7. Create a Bedrock Guardrail with content filtering for responsible AI.

Another skill I want to learn is to develop more different applications using Amazon Bedrock.